

COMP0171 Bayesian Deep Learning

1. Detailed balance. $p^*(x) T(x \rightarrow x') = p^*(x') T(x' \rightarrow x)$. p^* is the invariant distribution.

$$A(x \rightarrow x') = \min \left(1, \frac{p(y, x') q(x' \rightarrow x)}{p(y, x) q(x \rightarrow x')} \right) \quad T(x \rightarrow x') = q(x \rightarrow x') A(x \rightarrow x').$$

2. Regularizers $r(w) = \sum_{d=1}^D |w_d|^q$ $q \rightarrow 0$, counting regularizer, i.e. # non-zero entries

3. Similar equivalency for L_1 regularization, place Laplace distribution prior.

$$p(w_d) = \frac{1}{2b} \exp\left(-\frac{|w_d - \mu|}{b}\right).$$

4. binary cross-entropy loss. $\sum_{i=1}^N y_i \log f(x_i, w) + (1 - y_i) \log (1 - f(x_i, w))$.

5. Multi-class classification: logits $z_i = w \phi(x_i)$ $\hat{y}_{i,k} = \text{soft max}(z_i) = \frac{\exp(z_{i,k})}{\sum_j \exp(z_{i,j})}$
 $\ell(\hat{y}_i, y_i) = -\sum_{j=1}^K y_{ij} \log \hat{y}_{ij}$

6. Stochastic gradient descent.

$$L(\theta) = \frac{1}{N} \sum_{i=1}^N L_i(\theta) = \frac{1}{N} \sum_{i=1}^N \ell(f(x_i; \theta), y_i). \quad \theta_{t+1} = \theta_t - \epsilon \nabla L_i(\theta).$$

7. Mini-batch $\nabla_{\theta} L(\theta) = \frac{1}{N} \sum_{i=1}^N \nabla_{\theta} L_i(\theta) \approx \frac{1}{M} \sum_{j=1}^M \nabla_{\theta} L_j(\theta)$.
 { Small batch $\rightarrow$ faster parameter update.
 { Large batch $\rightarrow$ less noisy estimate, faster convergence.

8. Moving average: $\underline{a}_t = \frac{1}{t} \sum_{\tau=1}^t x_{\tau} = \frac{1}{t} (x_t + (t-1) \underline{a}_{t-1}) = \epsilon_t x_t + \mu_t \underline{a}_{t-1}$.

If μ_t small then more recent x contribute more strongly to the moving average.

9. Momentum: $\tilde{g}_{t+1} = \mu_t \tilde{g}_t - \epsilon \nabla f(\theta_t)$ $\theta_{t+1} = \theta_t + \tilde{g}_{t+1}$

10. Adagrad: different learning rate in every direction.

$$g_{t,i} = (\nabla L(\theta^t))_i \quad G_{t,i} = \sum_{\tau=1}^t g_{\tau,i}^2$$

$$\theta_i^{t+1} = \theta_i^t - \frac{\eta}{\sqrt{G_{t,i}} + \epsilon} g_{t,i}.$$

gradient $\uparrow$, $g_{t,i} \uparrow$, $G_{t,i} \uparrow$, learning rate $\downarrow$
 bad: accumulated gradients eventually make learning very slow and never recover.

11. RMSprop: moving average of past squared gradients.

$$S_{t,i} = \gamma S_{t-1,i} + (1-\gamma) g_{t,i}^2 \quad \theta_i^{t+1} = \theta_i^t - \frac{\eta}{\sqrt{S_{t,i}} + \epsilon} g_{t,i}.$$

12. Adam: RMSprop + momentum

$$m_{t,i} = \beta m_{t-1,i} + (1-\beta) g_{t,i} \quad \hat{m}_{t,i} = \frac{m_{t,i}}{1-\beta^t}$$

$$s_{t,i} = \gamma s_{t-1,i} + (1-\gamma) g_{t,i}^2 \quad \hat{s}_{t,i} = \frac{s_{t,i}}{1-\gamma^t}$$

$$\theta_i^{t+1} = \theta_i^t - \frac{\eta}{\sqrt{\hat{s}_{t,i}} + \epsilon} \hat{m}_{t,i}.$$

} bias corrected.

13. Quasi-Newton methods: find B satisfying $\nabla f(\theta_{t+1}) = \nabla f(\theta_t) + B(\theta_{t+1} - \theta_t)$.
 $\theta_{t+1} = \theta_t - \epsilon B_t^{-1} \nabla f(\theta_t)$. superlinear convergence order.

14. We can also construct $B = \lambda \sum_{n=1}^N \nabla f_n(\theta) \nabla f_n(\theta)^T$ for minimizing $L(\theta) = \frac{\lambda}{2} \sum_{n=1}^N (y_n - f(\theta, x_n))^2$.

15. Autodiff - Forward mode: low memory, easy to implement, full gradient $O(df)$

- Backward mode: memory intensive, run time is $O(f)$

16. Dual arithmetic: $\epsilon^2 = 0$, $f(x) = \text{DualPart}(f(x))$ Dualpart is coefficient of ϵ .

$$\sin(v_1 + \epsilon) = \sin(v_1) + \cos(v_1) \epsilon.$$

$$f(v_1, v_2) = v_1 v_2 - \sin(v_2) \quad f(v_1 + \epsilon, v_2) = (v_1 + \epsilon) v_2 - \sin(v_2) = \underbrace{v_1 v_2 - \sin(v_2)}_{f(v_1, v_2)} + \underbrace{v_2 \epsilon}_{\frac{\partial f}{\partial v_1}}.$$

$$f(v_1, v_2 + \epsilon) = v_1 (v_2 + \epsilon) - \sin(v_2 + \epsilon) = v_1 v_2 + v_1 \epsilon - \sin(v_2) - \cos(v_2) \epsilon = \underbrace{v_1 v_2 - \sin(v_2)}_{f(v_1, v_2)} + \underbrace{(v_1 - \cos(v_2)) \epsilon}_{\frac{\partial f}{\partial v_2}}.$$

$$f(x + \epsilon) = f(x) + f'(x) \epsilon$$

17. Forward: $\dot{u}_{n+1} = \frac{\partial u_{n+1}}{\partial u_0} = \frac{\partial u_{n+1}}{\partial u_n} \cdot \frac{\partial u_n}{\partial u_0} = \tilde{D}_n \cdot \dot{u}_n \Rightarrow \dot{u}_{n+1} = \tilde{D}_n \dot{u}_n$ good when #input << #outputs

Backward: $\dot{v}_n^T = \frac{\partial f}{\partial v_n} = \frac{\partial f}{\partial v_{n+1}} \frac{\partial v_{n+1}}{\partial v_n} = \dot{v}_{n+1}^T \cdot D_n \Rightarrow \dot{v}_n = D_n \dot{v}_{n+1}$ good when #input >> #outputs

18. Minibatches with priors.

$$\mathcal{L}(w) = \log p(w) + \sum_{i=1}^N \log p(y_i | w, x_i) \quad \mathcal{L}_M(w) = \log p(w) + \frac{N}{M} \sum_{j=1}^M \log p(y_j | w, x_j)$$

19. Laplace approximation: $\log p(w|D) \approx \log \mathcal{N}(w|w^*, \Lambda^{-1}) = -\frac{1}{2}(w-w^*)^T \Lambda (w-w^*) + \text{const}$
 $w^* = w_{\text{map}}, \quad \Lambda = -H = -\nabla \nabla \log p(w|D)|_{w=w^*}$

20. ELBO: $\mathbb{E}_{q_\lambda(w)} [\log \frac{p(w,D)}{q_\lambda(w)}] = \log p(D) - \text{KL}[q_\lambda(w) || p(w|D)]$.

$$\nabla_\lambda \mathbb{E}_{q_\lambda(w)} [\log p(w,D) - \log q_\lambda(w)] = \mathbb{E}_{q_\lambda(w)} [\log \frac{p(w,D)}{q_\lambda(w)} \nabla_\lambda \log q_\lambda(w)]$$

21. Reparametrisation trick: $\varepsilon \sim \mathcal{N}(0, I) \quad w = \mu + \Sigma^{\frac{1}{2}} \varepsilon$
 $\nabla_\lambda \mathbb{E}_{q_\lambda(w)} [\log \frac{p(w,D)}{q_\lambda(w)}] = \nabla_\lambda \mathbb{E}_{p(\varepsilon)} [\log \frac{p(r(\lambda, \varepsilon), D)}{q_\lambda(r(\lambda, \varepsilon))}] = \mathbb{E}_{p(\varepsilon)} [\nabla_\lambda \log \frac{p(r(\lambda, \varepsilon), D)}{q_\lambda(r(\lambda, \varepsilon))}]$

22. Transforming parameters: for $\Sigma = \text{diag}(\sigma^2)$, can use $\sigma_z^2 = T(z)$ $T(z) = \exp(z)$ or $T(z) = \log(1 + \exp(z))$.
 for arbitrary Σ , consider $\Sigma = AA^T + \text{diag}(\sigma^2)$ (low rank + diag).

23. $\mathbb{E}_{q_\lambda(w)} [\log \frac{p(w,D)}{q_\lambda(w)}] = \sum_{i=1}^N \mathbb{E}_{q_\lambda(w)} [\log p(y_i | x_i, w)] - \text{KL}[q_\lambda(w) || p(w)]$
 $\approx \frac{N}{M} \sum_{j=1}^M \mathbb{E}_{p(\varepsilon)} [\log p(y_j | x_j, r(\lambda, \varepsilon))] - \text{KL}[q_\lambda(w) || p(w)]$

24. $\text{KL}[q_\lambda || p]$ has mode seeking or zero-forcing behaviour

$$\mathbb{E}_{q_\lambda(z)} [\log \frac{q_\lambda(z)}{p(z|D)}] = \mathbb{E}_{q_\lambda(z)} [\log q_\lambda(z)] - \mathbb{E}_{q_\lambda(z)} [\log p(z|D)]$$

If $p(z|D) \rightarrow 0$ at some value but $q_\lambda(z) > 0$, then this cross entropy will be $-\infty$, the whole KL $\rightarrow +\infty$.

25. $\text{KL}[p || q_\lambda]$ has mass-covering or mean-seeking behaviour

$$\mathbb{E}_{p(z|D)} [\log \frac{p(z|D)}{q_\lambda(z)}] = \mathbb{E}_{p(z|D)} [\log p(z|D)] - \mathbb{E}_{p(z|D)} [\log q_\lambda(z)]$$

If $q_\lambda(z) \rightarrow 0$, $p(z|D) > 0$, this KL $\rightarrow \infty$
 So wherever $p(z|D)$ has positive mass, $q_\lambda(z)$ must be positive.

26. Sequential learning: If you add data points sequentially, old posterior is new prior.

27. Latent is discrete:

- score function estimators $\frac{\partial}{\partial \theta} \mathbb{E}_{p(z|\theta)} [f(z, \theta)] = \frac{\partial}{\partial \theta} \sum_z f(z, \theta) p(z|\theta) = \sum_z \frac{\partial f}{\partial \theta} p(z|\theta) + f(z, \theta) \frac{\partial p(z|\theta)}{\partial \theta}$
 (may requires much larger L)
 $= \sum_z \frac{\partial f}{\partial \theta} p(z|\theta) + f(z, \theta) p(z|\theta) \frac{\partial \log p(z|\theta)}{\partial \theta}$
 $= \mathbb{E}_{p(z|\theta)} [\frac{\partial f}{\partial \theta} + f(z, \theta) \frac{\partial \log p(z|\theta)}{\partial \theta}]$
 $\approx \frac{1}{L} \sum_{\ell=1}^L [\frac{\partial f}{\partial \theta} + f(z_\ell, \theta) \frac{\partial \log p(z_\ell|\theta)}{\partial \theta}]$

- Explicit marginalization eg: z is Bernoulli $\frac{\partial}{\partial \theta} \mathbb{E}_{p(z|\theta)} [f(z, \theta)] = \sum_{z \in \{0,1\}} p(z|\theta) \frac{\partial f(z, \theta)}{\partial \theta} + f(z, \theta) \frac{\partial p(z|\theta)}{\partial \theta}$
 (z space needs small).

28. Latent contains discrete and continuous, marginalize discrete and do ELBO on continuous.

$$\log p(D, z_c) = \log \sum_{z_d} p(D, z_c, z_d)$$

normaliser by sum of all $z_d = k \forall k$.

$$p(z_d = k | D) = \int p(z_d = k | z_c, D) p(z_c | D) dz_c \approx \frac{1}{O} \int p(z_d = k, z_c, D) q_\lambda(z_c) dz_c$$

29. Marginal likelihood $p(D|\mu)$ for model comparison.

30. Model selection: with Laplace approximation

$$\log p(D, \theta) \approx \log p(D, \theta^*) - \frac{1}{2}(\theta - \theta^*)^T \Lambda (\theta - \theta^*)$$

$$\Lambda = -H = -\nabla \nabla \log p(D, \theta)|_{\theta=\theta^*}$$

Occam factor: penalising model complexity

$$p(D) = \int p(D|\theta) d\theta \approx \int p(D|\theta^*) p(\theta^*) \exp(-\frac{1}{2}(\theta - \theta^*)^T \Lambda (\theta - \theta^*)) d\theta$$

$$= p(D|\theta^*) p(\theta^*) \frac{(2\pi)^{\frac{D}{2}}}{|\Lambda|^{\frac{1}{2}}}$$

$$\log p(D) \approx \log p(D|\theta^*) + \log p(\theta^*) + \frac{D}{2} \log 2\pi - \frac{1}{2} \log |\Lambda|$$

Occam factor

prefer broader rather than peaky posteriors.
 broader $\rightarrow$ large determinant of Cov matrix

31. Deep learning: $f(x) = w^T \Phi(x)$ learn w and Φ .

32. Simplest deep model: $f(x) = w^T g(Ax+b)$. $g(\cdot)$ is non linear.

33. Multi-layer perceptron / feed forward network:

$$y = w^T h_1, h_1 = g(A_1 h_2 + b_1), h_2 = g(A_2 h_3 + b_2) \dots h_L = g(A_L x + b_L).$$

34. Loss function for deep learning:

- Squared error $\leftrightarrow$ Gaussian log likelihood.
- Binary cross entropy / log loss $\leftrightarrow$ Bernoulli log likelihood.
- Absolute error $\leftrightarrow$ Laplace log likelihood

35. Regularization: - adding L_2 or L_1 norm $\| \theta \|_2$ - early stopping.

36. Bayesian last layer: $p(y|x) = p(y | w^T g(A_L g(\dots x) + b_L))$ - find point estimates for $\{A_L, b_L\}_{L=1}^L$ and estimate a posterior $p(w|D)$

37. deep ensembles: find S different point estimates for $w^s, \{A^s, b^s\}_{L=1}^L$ from different initialisations and treat them as a set of "samples".

38. Many Bayesian inference algorithms don't deal with disconnected modes well.

- Laplace approximation
- Variational Bayes
- MCMC.

Ensembles capture "global" variation.

39. Deep learning learns representations. Bayesian deep learning learns distributions over representations.

40. Convolutional layers: ① locality ② Translation invariant (share same filter weights across every spatial location).

41. Convolution operation: $(f * g)(t) = \int f(a) g(t-a) da$ $f * g = g * f$
slide $g(-a)$ around real line, compute overlapped area.

discrete case: $(x * w)_d = \sum_m x_m w_{d-m}$ 2D case $(X * W)_{i,j} = \sum_m \sum_n x_{m,n} w_{i-m, j-n}$
 $x \in \mathbb{R}^D$ $w \in \mathbb{R}^M$ $M < D$.

42. Convolutional block:

- Convolutional stage: affine transform
- Detector stage: nonlinearity, eg rectified linear.
- Pooling stage. (replace a spatial region of the output with a summary statistic) max pooling, average pooling

43. Residual connections or skip connections

$h_1 = \underbrace{M(h_2)}_{\text{this could have vanishing gradients}} + A h_2$ make easier for gradients to flow back to deeper layers.

44. Batch Norm: Batch Norm layer rescales the input x to have unit mean and standard deviation.

$$\mu_d = \frac{1}{M} \sum_{i=1}^M x_d^i \quad \sigma_d^2 = \frac{1}{M} \sum_{i=1}^M (x_d^i - \mu_d)^2 \quad \hat{x}_d^i = \frac{x_d^i - \mu_d}{\sqrt{\sigma_d^2 + \epsilon}} \quad y_d^i = \gamma_d \hat{x}_d^i + \beta_d$$

we have M data in a batch. We do normalisation over batch, not feature dimension!

resulting vector $y = \text{BatchNorm}(x)$ where γ_d, β_d are trainable parameters.

45. $p(y|x, \theta^*) = \text{Categorical}(y | f_{\theta^*}(x))$ For a 100-class classification problem, suppose $\hat{y} = f_{\theta^*}(x) \in [0, 1]^{100}$

with $\sum_{k=1}^{100} \hat{y}_k = 1$ eg. $\hat{y} = [0.03, 0.1, \dots, 0.003, 0.5]^T$ $\hat{y}_k = \Pr(y=k)$ $\hat{y}_{\text{label}} = \arg \max_k \hat{y}_k$

A well-calibrated classifier would do $\Pr(\hat{y}_{\text{label}} = y | \hat{p} = p) = p$ eg. $\hat{y}_{\text{label}} = 100, p_{\text{out}} = 0.5, \Pr(\hat{y}_{\text{label}} = y | \hat{p} = 0.5) = 0.5$
consider test points i have $\hat{p}_i \in [0.3, 0.4]$, we expect between 30% to 40% of those points have been classified correctly.

46. Estimating uncertainty calibration:

M intervals $I_m = [\frac{m-1}{M}, \frac{m}{M}]$ partitioning $[0,1]$, let $B_m = \{i: \hat{p}_i \in I_m\}$ bin of indices whose confidence falls in the interval I_m .

Accuracy: $\text{acc}(B_m)$ is $\frac{1}{|B_m|} \sum_{i \in B_m} \underbrace{1\{\hat{y}_i = y_i\}}_{\text{true frequency}}$ Average confidence: $\text{conf}(B_m)$ is $\frac{1}{|B_m|} \sum_{i \in B_m} \underbrace{\hat{p}_i}_{\text{predicted prob.}}$

A well calibrated model should have $\text{acc}(B_m) = \text{conf}(B_m)$ for $\forall m$.

Expected Calibration Error $\text{ECE} = \sum_{m=1}^M \frac{|B_m|}{N} |\text{acc}(B_m) - \text{conf}(B_m)| = \frac{1}{N} \sum_{m=1}^M \left| \sum_{i \in B_m} 1\{\hat{y}_i = y_i\} - \hat{p}_i \right|$.

47. Temperature scaling: won't change class prediction, change the confidence (probabilities)

48. $p(y|x, D) = \int \underbrace{p(y|x, \mu)}_{\text{aleatoric uncertainty}} \underbrace{p(\mu|D)}_{\text{epistemic uncertainty}} d\mu$

Aleatoric: irreducible uncertainty in the measurement process. Even if we know which model is correct, we still have such probability (uncertainty).

Epistemic: uncertainty about correct model given limited data.

More data $\rightarrow$ less epistemic uncertainty

49. $\mathcal{N}(y|\mu, \sigma^2 I)$, $Z = \{\mu, \sigma\}$

$$\begin{aligned} \text{Var}(y) &= \text{Var}[\mathbb{E}[y|Z]] + \mathbb{E}[\text{Var}[y|Z]] \\ &= \underbrace{\text{Var}[\mu]}_{\text{epistemic}} + \underbrace{\mathbb{E}_{p(\sigma|D)}[\sigma^2 I]}_{\text{aleatoric}} \end{aligned}$$

Aleatoric can be reduced by having new features

Epistemic can be reduced by having more data.

50. Laplace approximation for posterior in deep learning:

$$\log p(\theta|D, \alpha, \beta) = \log p(\theta|\alpha) + \sum_{i=1}^N \log p(y_i|x_i, \theta, \beta) = -\frac{\alpha}{2} \theta^T \theta - \frac{\beta}{2} \sum_{i=1}^N (f_\theta(x_i) - y_i)^2 + \text{const}$$

$$\begin{aligned} \textcircled{1} \text{ find MAP on } \theta^* \quad \textcircled{2} \Lambda &= -\nabla \nabla \log p(\theta|D, \alpha, \beta) = \alpha I + \beta H, \quad H = \nabla \nabla \sum_{i=1}^N (f_\theta(x_i) - y_i)^2 \\ &= \sum_{i=1}^N \nabla f_\theta(x_i) (\nabla f_\theta(x_i))^T + \sum_{i=1}^N (f_\theta(x_i) - y_i) \nabla \nabla f_\theta(x_i) \\ &\approx \sum_{i=1}^N \nabla f_\theta(x_i) (\nabla f_\theta(x_i))^T \end{aligned}$$

③ make predictions $p(y|x, D) = \int p(y|x, \theta) q(\theta|D) d\theta$

- monte carlo $p(y|x, D) \approx \frac{1}{K} \sum \mathcal{N}(y|f_\theta(x), \beta^{-1}), \theta^k \sim q(\theta|D)$.

- or $f_\theta(x) \approx f_\theta^*(x) + g(x)^T (\theta - \theta^*)$, $g(x) = \nabla f_\theta(x)|_{\theta=\theta^*}$

$p(y|x, \theta, \beta) \approx \mathcal{N}(y|f_\theta^*(x) + g(x)^T (\theta - \theta^*), \beta^{-1})$.

$p(y|x) \approx \mathcal{N}(y|f_\theta^*(x), \underbrace{g(x)^T \Lambda^{-1} g(x)}_{\text{epistemic}} + \underbrace{\beta^{-1}}_{\text{Aleatoric}})$

51. Langevin dynamics: $\log p(\theta, D) = \sum \log p(y_i|x_i, \theta) + \log p(\theta)$.

$g_t = -\nabla_\theta \log p(\theta^t, D)$ and update $\theta^{(t+1)} = \theta^{(t)} - \eta g_t$ (gradient descent).

Langevin Monte Carlo propose $\theta' = \theta - \underbrace{\frac{1}{2} \Sigma^2 g}_{\eta} + \varepsilon p$ $p \sim \mathcal{N}(0, I)$.

can interpret as MH proposal $q(\theta \rightarrow \theta') = \mathcal{N}(\theta' | \theta - \eta g, \Sigma^2 I)$.

52. Stochastic gradient Langevin dynamics:

① Sample $p_t \sim \mathcal{N}(p|0, I)$

② Compute update $\theta^{(t+1)} = \theta^{(t)} + \frac{1}{\Sigma} \Sigma_t^2 \nabla \log \tilde{\pi}_M(\theta^{(t)}) + \Sigma_t p_t$ $\nabla \log \tilde{\pi}_M(\theta^{(t)})$ getting from mini-batch!

- skip MH acceptance ratio to avoid $\pi(\theta)$ evaluation, in $\Sigma \rightarrow 0$ case the proposal will be accepted for sure

Σ_t chosen to be cyclic learning rates for exploration (don't sample whenever it is large, that is exploration)

53. Local reparameterization trick: If $A \in \mathbb{R}^{H \times D}$, $b \in \mathbb{R}^H$ are Gaussian distributed,

$h = Ax + b \in \mathbb{R}^H$ is also Gaussian, sample h since it has lower dimension.

54. For data (x_i, y_i) . generative model: learn approximation to $p_\theta(x_i)$ or $p_\theta(x_i, y_i)$

discriminative model: learn $p_\theta(y_i|x_i)$.

55. Per data ELBO:

$$\log p_{\theta}(x_i) = \mathbb{E}_{q_{\phi}(z_i|x_i)} \left[\log \frac{p_{\theta}(x_i, z_i)}{q_{\phi}(z_i|x_i)} \right] + \text{KL}[q_{\phi}(z_i|x_i) \parallel p_{\theta}(z_i|x_i)]$$

$$\mathcal{L}(x; \phi, \theta) = \mathbb{E}_{q_{\phi}(z|x)} \left[\log \frac{p_{\theta}(x, z)}{q_{\phi}(z|x)} \right] = \underbrace{\mathbb{E}_{q_{\phi}(z|x)} [\log p_{\theta}(x|z)]}_{\text{reconstruction item}} - \underbrace{D_{\text{KL}}(q_{\phi}(z|x) \parallel p_{\theta}(z|x))}_{\text{regularization item}}. \quad \text{estimate } \theta \text{ and } \phi.$$

Gradients of ∇_{θ} :

$$\nabla_{\theta} \mathbb{E}_{q_{\phi}(z|x)} [\log p_{\theta}(x|z)] = \mathbb{E}_{q_{\phi}(z|x)} [\nabla_{\theta} \log p_{\theta}(x|z)] \quad \text{how well of the reconstruction from } x \rightarrow z \rightarrow x?$$

Gradients of ∇_{ϕ} : reparametrisation trick

$$z \sim p(z) \quad \tilde{z} = g_{\phi}(z, x) \quad \text{and then } \tilde{z} \sim q_{\phi}(z|x).$$

$$\nabla_{\phi, \theta} \mathbb{E}_{q_{\phi}(z|x)} [\log p_{\theta}(x|z)] = \mathbb{E}_{p(z)} [\nabla_{\phi, \theta} \log p_{\theta}(x|\tilde{z} = g_{\phi}(z, x))].$$

$$\text{If } p_{\theta}(x|z) \text{ is Gaussian, } \log p_{\theta}(x|\tilde{z} = g_{\phi}(z, x)) = -\frac{1}{2\sigma^2} \|x - f_{\theta}(g_{\phi}(z, x))\|_2^2 + \text{const}$$

recover PPCA if f_{θ} and g_{ϕ} are linear functions.

56. Joint generative model: $p_{\theta}(x, y, z) = p_{\theta}(x|y, z) p(z) p(y)$

$$p(z) = \mathcal{N}(z|0, I)$$

$$p(y) = \text{Discrete}(y|\pi)$$

$$p_{\theta}(x|y, z) = f(x; y, z, \theta).$$

Approximate posterior:

$$q(y|x) = \text{Discrete}(y|\pi_{\phi}(x))$$

$$q(z|x, y) = \mathcal{N}(z|\mu_{\phi}(x, y), \text{diag}(\sigma_{\phi}(x, y)^2)).$$

57. Semi-supervised learning:

N^S image, label pairs (x_i, y_i) N^U unlabelled x_j .

$$\text{Supervised case: } \mathcal{L}(x, y; \phi, \theta) = \mathbb{E}_{q_{\phi}(z|x, y)} [\log p_{\theta}(x, y, z) - \log q_{\phi}(z|x, y)].$$

$$\text{Unsupervised case: } \mathcal{U}(x; \phi, \theta) = \mathbb{E}_{q_{\phi}(y|x)} \mathbb{E}_{q_{\phi}(z|x, y)} [\log p_{\theta}(x, y, z) - \log q_{\phi}(y|x) - \log q_{\phi}(z|x, y)].$$

$$\mathcal{J} = \sum_{i=1}^{N^S} \mathcal{L}(x_i, y_i; \phi, \theta) + \sum_{j=1}^{N^U} \mathcal{U}(x_j; \phi, \theta) + \alpha \sum_{i=1}^{N^S} \log q_{\phi}(y_i|x_i) \quad (\text{otherwise } \pi_{\phi}(x) \text{ is only estimated from unsupervised data}).$$

58. Single hidden layer full Bayesian:

A network with one hidden layer of width H . For an input $x_i \in \mathbb{R}^D$, the output is

$$f(x_i) = a + w^T g(b + V^T x_i)$$

$V \in \mathbb{R}^{D \times H}$ has columns v_h , $g(\cdot)$ non-linear function

$$a \sim \mathcal{N}(0, \sigma_a^2) \quad w_h \sim \mathcal{N}(0, \sigma_w^2)$$

$w \in \mathbb{R}^H$ are second layer weights, $a \in \mathbb{R}$, $b \in \mathbb{R}^H$

$$b_h \sim p(b) \quad v_h \sim p(v) \quad \text{all independent}$$

$$\mathbb{E}[f(x)] = \mathbb{E}[a] + \sum_{h=1}^H \mathbb{E}[w_h] \mathbb{E}[g(b + v_h^T x)] = 0$$

$$\begin{aligned} \mathbb{E}[f(x) f(x')] &= \mathbb{E}[a^2] + \sum_{h=1}^H \mathbb{E}[w_h^2] \mathbb{E}_{b, v} [g(b + v^T x) g(b + v^T x')] \\ &= \sigma_a^2 + \sum_{h=1}^H \sigma_w^2 \mathbb{E}_{b, v} [g(b + v^T x) g(b + v^T x')]. \end{aligned}$$

$$\text{If we set } \sigma_w^2 = \frac{S^2}{H}$$

$$\text{cov}(f(x), f(x')) = \sigma_a^2 + S^2 \mathbb{E}_{b, v} [g(b + v^T x) g(b + v^T x')] = k(x, x')$$

x is iid, as $H \rightarrow \infty$ $w^T g(b + V^T x_i) \rightarrow \text{gaussian}$ so $f(\cdot) \sim \mathcal{GP}(0, k(\cdot, \cdot))$.

59. In gaussian case, entropy depends on only variance.

60. Mutual Information: $I(X; Y) = \text{KL}[P_{XY} \parallel P_X P_Y] = H[X] - \underbrace{H[X|Y]}_{\text{entropy on conditional distribution}} = H[Y] - H[Y|X]$

61. Active learning:

labelled set: $\{(x_i, y_i)\}_{i=1}^N = \mathcal{D}$, Unlabelled pool: $\mathcal{U} = \{x_j\}_{j=1}^M$, $M \gg N$

Assume current model gives a predictive distribution $P(y|x) = [\alpha_1, \dots, \alpha_K]$, $\alpha_k = P(y=k|x)$.

$J(x) = \max_{\alpha_k} P(y|x)$. we choose to pick x^* such that $x^* = \arg\min_{x \in \mathcal{U}} J(x)$. (pick the point least certain).

can also do $\arg\max_{x \in \mathcal{U}} H(P(y|x))$ maximum entropy

62. Bayesian active learning by disagreement (BALD)

Which x^* will most reduce our uncertainty about parameter θ .

Mutual information between unknown label y at x and parameter θ

$$\begin{aligned} \max I(y; \theta | \mathcal{D}, x) &= \text{KL}[P(y; \theta | \mathcal{D}, x) \| P(y | \mathcal{D}, x) P(\theta | \mathcal{D}, x)] = \mathbb{E}_{P(y; \theta | \mathcal{D}, x)} \left[\log \frac{P(y; \theta | \mathcal{D}, x)}{P(y | \mathcal{D}, x) P(\theta | \mathcal{D}, x)} \right] \\ &= \mathbb{E}_{P(y|x, \theta) P(\theta | \mathcal{D})} \left[\log \frac{P(y|x, \theta) P(\theta | \mathcal{D})}{P(y | \mathcal{D}, x) P(\theta | \mathcal{D})} \right] = \int_{\theta} \int_y P(\theta | \mathcal{D}) P(y|x, \theta) \log \frac{P(y|x, \theta)}{P(y | \mathcal{D}, x)} dy d\theta \\ &= - \int_y \left[\int_{\theta} P(\theta | \mathcal{D}) P(y|x, \theta) d\theta \right] \log P(y | \mathcal{D}, x) dy + \int_{\theta} P(\theta | \mathcal{D}) \int_y P(y|x, \theta) \log P(y|x, \theta) dy d\theta \\ &= \underbrace{H[y | \mathcal{D}, x]}_{\text{picks the point model is currently confused}} - \underbrace{\mathbb{E}_{\theta \sim P(\theta | \mathcal{D})} [H[y|x, \theta]]}_{\text{average per-model uncertainty}} \end{aligned}$$

If all posterior samples θ agree on the label, $H[y | \mathcal{D}, x]$ is low, learning y won't tell you much about which θ is right, low I .

If different θ give very different predictive distributions, $P(y | \mathcal{D}, x)$ may still be high-entropy, but each $H[y|x, \theta]$ is low, high I .

63. Bayesian Optimisation:

- evaluate $f(x)$ is very costly, - gradient of f are unavailable - limit budget of evaluation.

① get initial samples

② Fit a model to the data $P(y|x, \mathcal{D}_n)$ most common Gaussian Process.

③ Select data collection strategy

acquisition function $\alpha(x) = \mathbb{E}_{P(y|x, \mathcal{D}_n)} [U(y|x, \mathcal{D}_n)]$

④ Optimize $\alpha(x)$

⑤ collect $\arg\max_x$ to update model

⑥ repeat.

Two choices of acquisition function:

- Expected Improvement (EI): $\alpha_{EI}(x') = \mathbb{E}_{P(\theta | \mathcal{D})} [\max(0, f(x'; \theta) - y^*)]$ $y^* = \max_{i \leq n} f(x_i)$.

- Entropy-search (ES): $\alpha_{ES}(x') = H(x^* | \mathcal{D}) - H(x^* | \mathcal{D} \cup \{x', y'\})$ $x^* = \arg\max_x f(x)$
(pick x' to maximise shrink the uncertainty about location of $\arg\max$).

64. Wake-sleep algorithm:

- Wake phase: update θ with real data, draw samples from $q_{\phi}(z|x)$, use them to update θ by $\min_{\theta} \text{KL}[q_{\phi} \| p_{\theta}]$

- Sleep phase: use learned model, sample x, z from $p_{\theta}(x, z)$, update ϕ by $\min_{\phi} \mathbb{E}_{p_{\theta}(x, z)} [\text{KL}[p_{\theta} \| q_{\phi}]]$.

65. Wake-wake: let $\hat{p}(x)$ data distribution, $\min \mathbb{E}_{\hat{p}(x)} [\text{KL}[p_{\theta} \| q_{\phi}]] = \mathbb{E}_{\hat{p}(x)} [\mathbb{E}_{p_{\theta}(z|x)} [-\log q_{\phi}]] + \text{const}$
importance sampling $w(z) = \frac{p_{\theta}(x, z)}{q_{\phi}(z|x)}$